ARTIFICIAL INTELLIGENCE
lecture 4

Informed Search

Uninformed search algorithms are systematic but inefficient.

They just repeat a cycle of:

- Choosing the next node to expand.
- Check whether it is the goal.
- If it is not, expand the node.

Nodes are chosen in a specific order (imposed by a search strategy), which is not necessarily the ”best” order in terms of getting closer/faster to the goal.

Informed Search

- Would like to use additional knowledge so that ”better” nodes are expanded/explored first.
- In terms of pseudo-code, we want to order the frontier so that ”better” nodes get popped first.
- We will introduce an evaluation function f (n) that indicates the desirability of considering node n next for exploration and expansion.
- Nodes with a better f (n) are always considered first.
- This approach is called best-first search.
- How should we compute f (n) ?

Informed Search

- Uniform-cost search (UCS) orders the frontier according to the path cost g(n):
  - Path cost is the distance from root to state n.
- So, uniform-cost search uses an evaluation function

  f (n) = g(n).
- The path cost g(n) only accounts for cost to reach n.
- Hence, uniform-cost search is not goal directed /oriented.

Informed Search

- Would also like to consider cost from node n to the goal.

More generally:

- We want the search algorithm to be able to estimate the path cost from the current node to the goal.
- This estimate is called a **heuristic function.**
- Cannot be done based on problem formulation:
- Need to add additional information.

Informed Search

Path cost g(n) is an exact value.

Heuristic function h(n):

- h(n): estimated cost from node n to goal.
- h(n1) < h(n2) means it is probably cheaper to get to the goal from n1.
- h(goal) = 0.

Evaluation function f (n):

- f (n) = g(n): Uniform-cost search.
- f (n) = h(n): Greedy best-first search.
- f (n) = g(n) + h(n): A* search.

Greedy best-first search

- GBFS algorithm always explores and expands the node judged to be closest to goal.
- It uses f (n) = h(n) and ignores the path cost g(n) entirely.
- Consider it to be the complement of uniform-cost search.
- GBFS algorithm is identical to UNIFORM-COST-SEARCH except that h is used instead of g.

Greedy best-first search(Tree version)

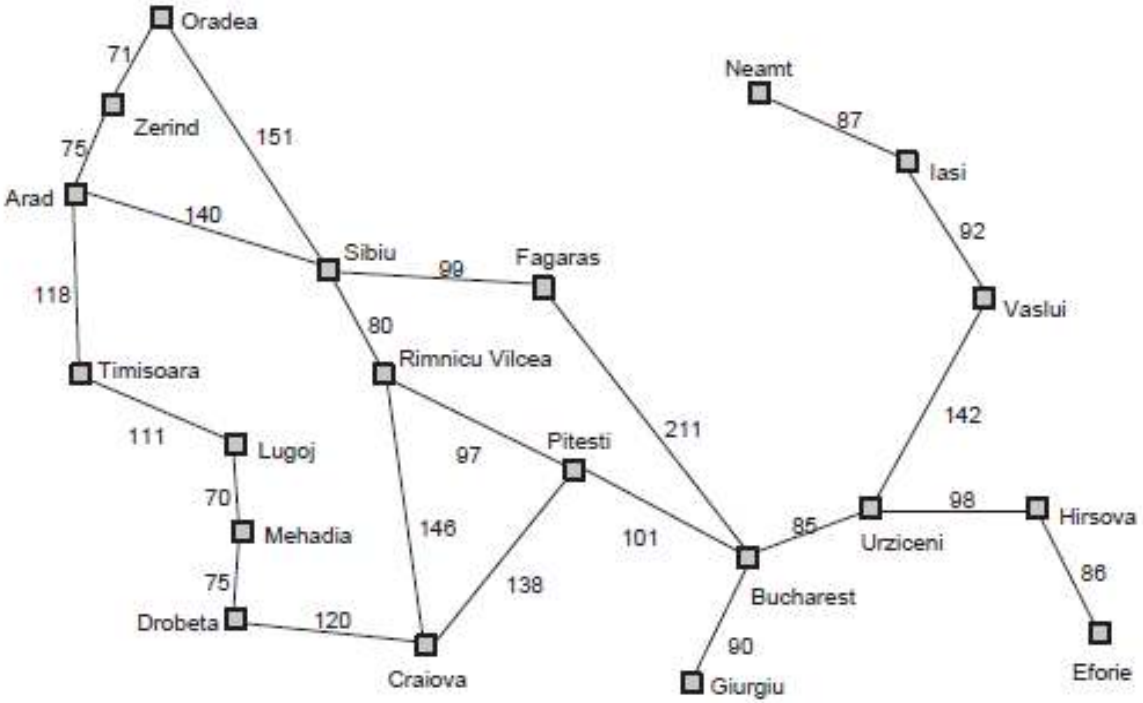
```
function GREEDY-BEST-FIRST-SEARCH(problem)
node ← STATE=problem:INITIAL-STATE, PATH-COST=0
frontier ← priority queue ordered by heuristic h
loop do
if frontier.EMPTY?() then return failure
node ← frontier.POP() /* choose lowest-cost node in frontier */
if problem.GOAL-TEST(node.STATE) then return node.SOLUTION()
for each action in problem.ACTIONS(node.STATE) do
child ← node.CHILD-NODE(problem, action)
Frontier.INSERT(child, h(child) )
```

Greedy best-first search(Graph version)

```
function GREEDY-BEST-FIRST-SEARCH(problem)
node ← STATE=problem:INITIAL-STATE, PATH-COST=0
frontier ← priority queue ordered by heuristic h.
explored ← an empty set
loop do
if frontier.EMPTY?() then return failure
node ← frontier.POP() /* choose lowest-cost node in frontier */
add node.STATE to explored
if problem.GOAL-TEST(node.STATE) then return node.SOLUTION()
for each action in problem.ACTIONS(node.STATE) do
child ← node.CHILD-NODE(problem, action)
if child.STATE is not in explored and not in frontier then
        Frontier.INSERT(child, h(child) )
else if child:STATE is in frontier with higher h-value then
 replace that frontier node with child
```

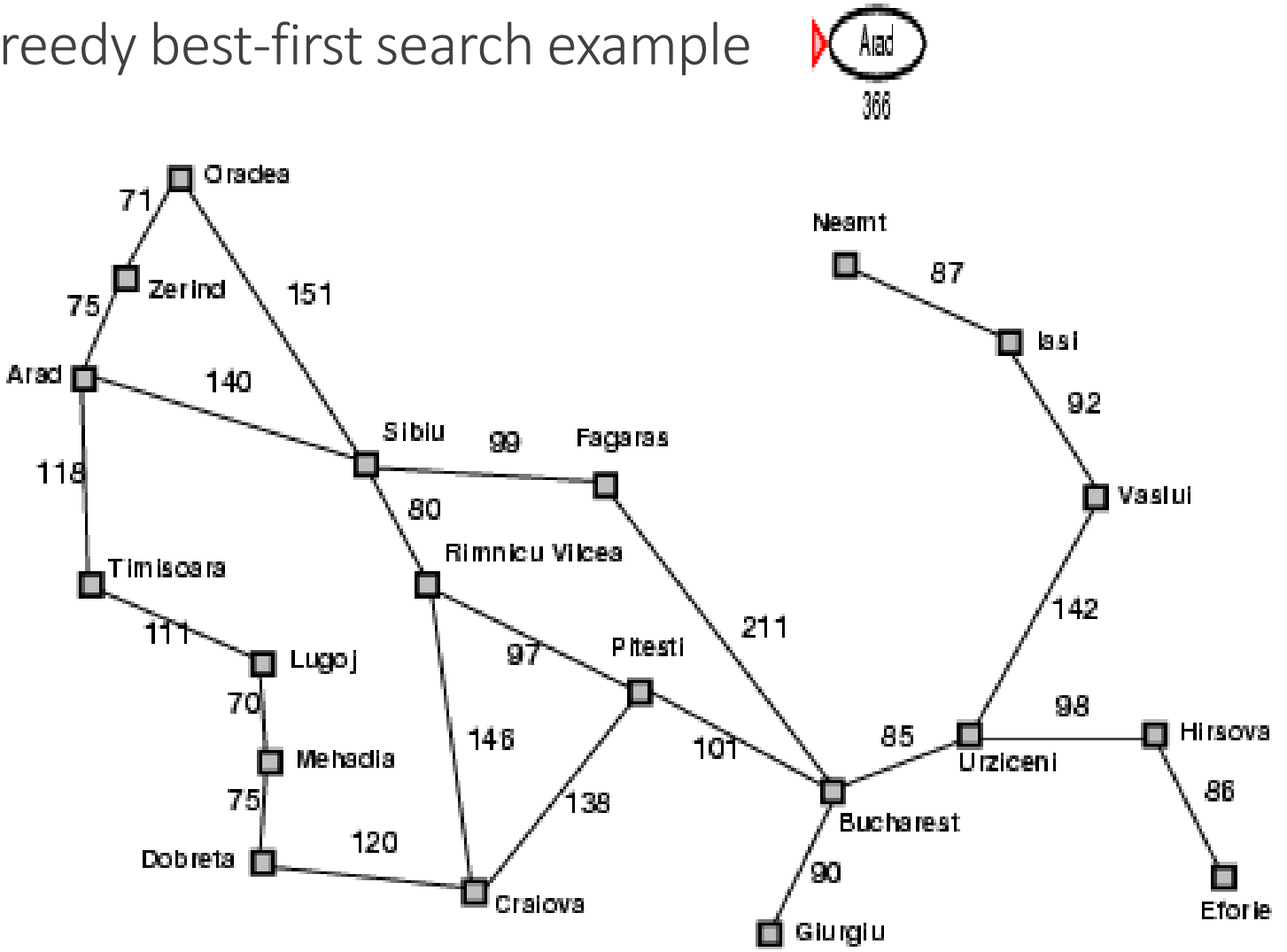
GBFS example - Romania map

Find a route from Arad to Bucharest.



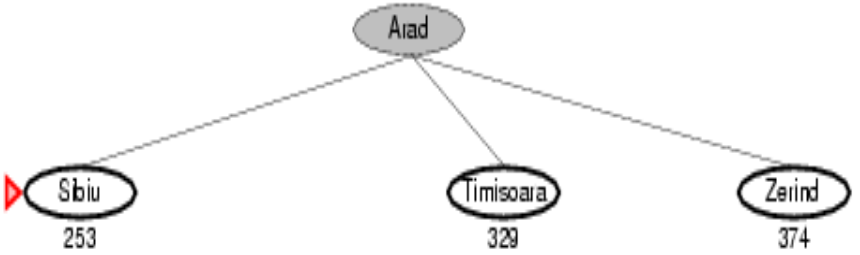
straight-line distances to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

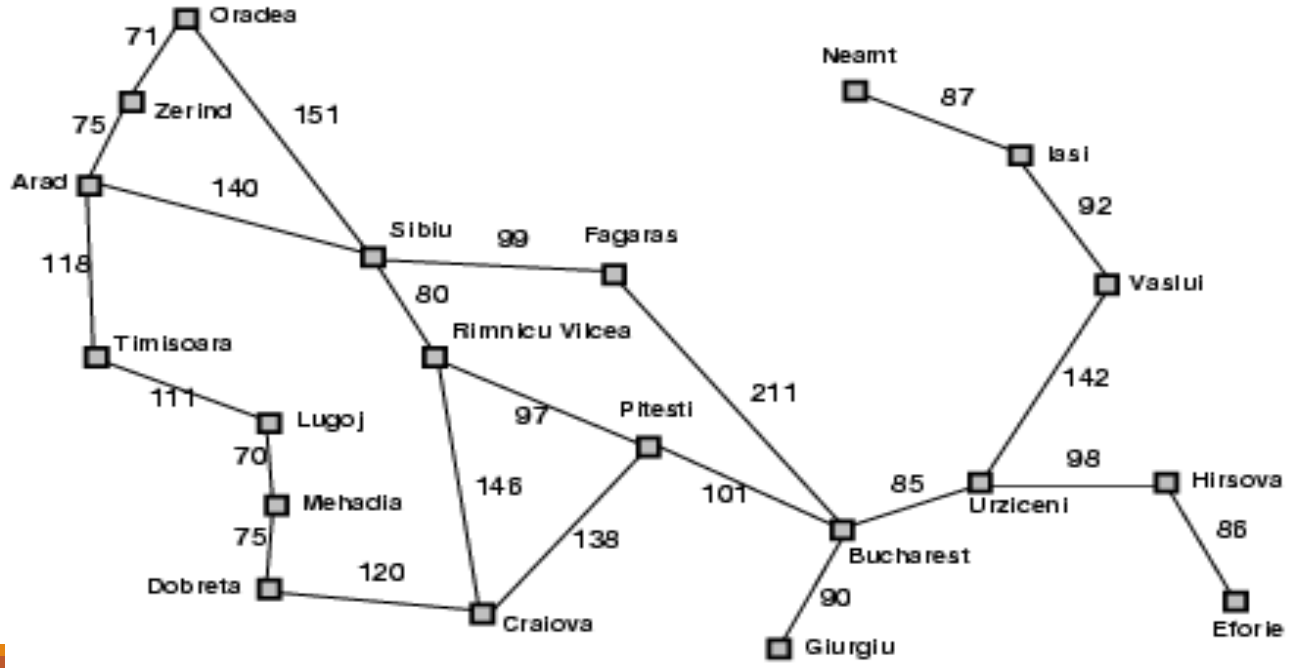
Greedy best-first search example



Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

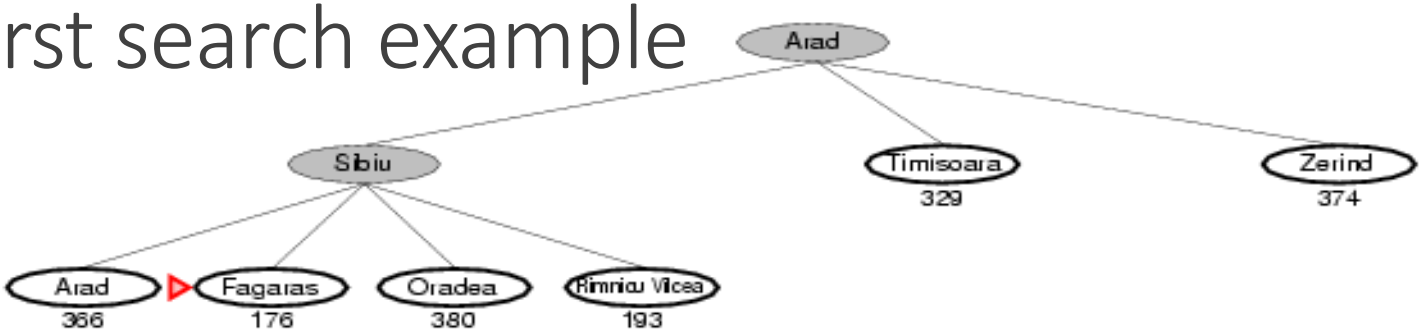
Greedy best-first search example

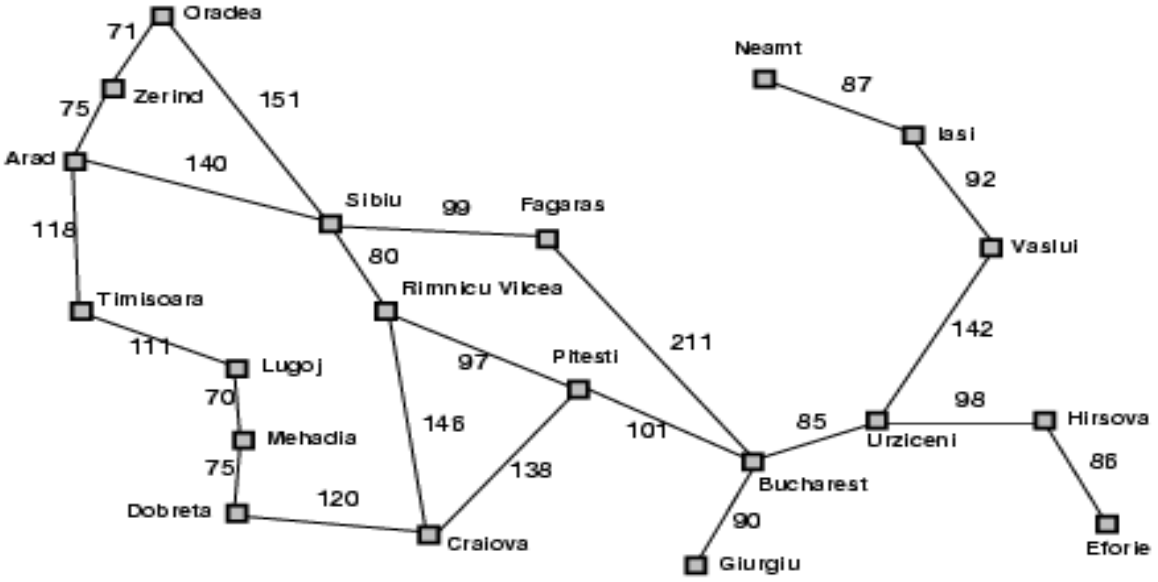




Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

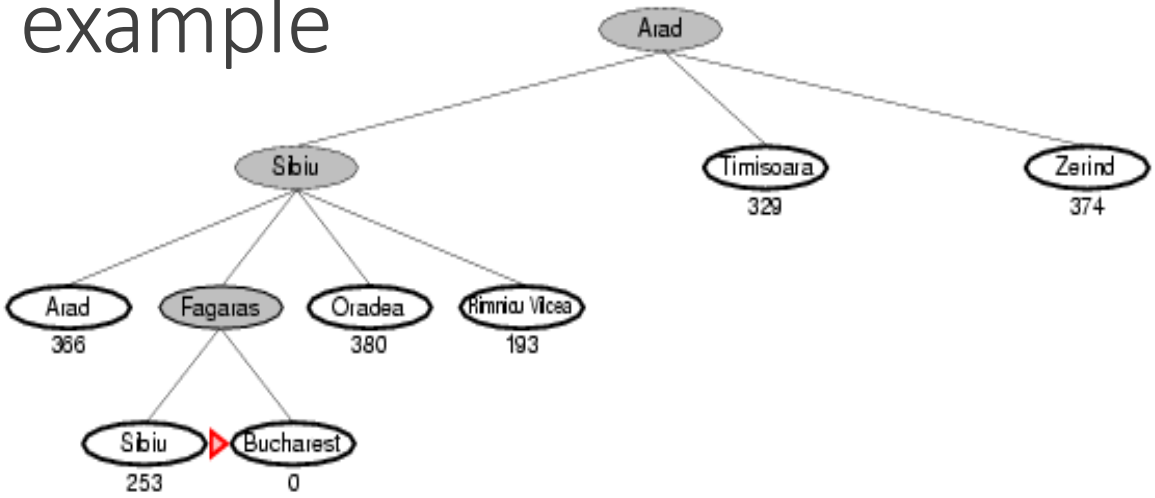
Greedy best-first search example

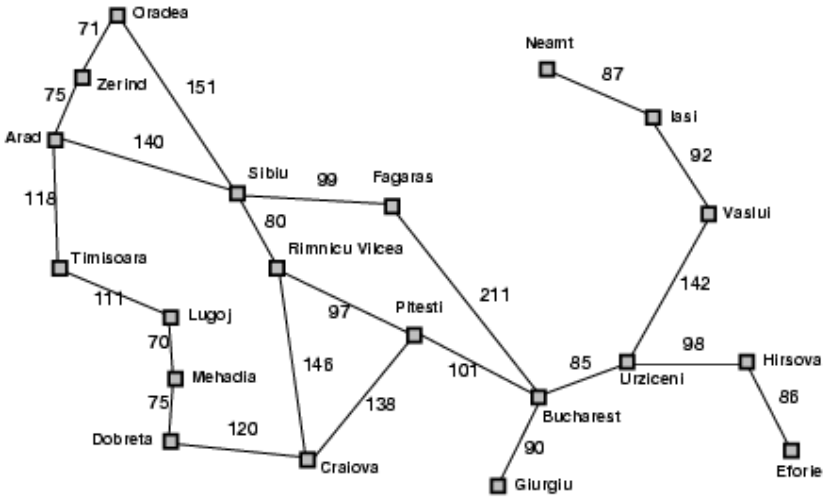




Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Greedy best-first search example

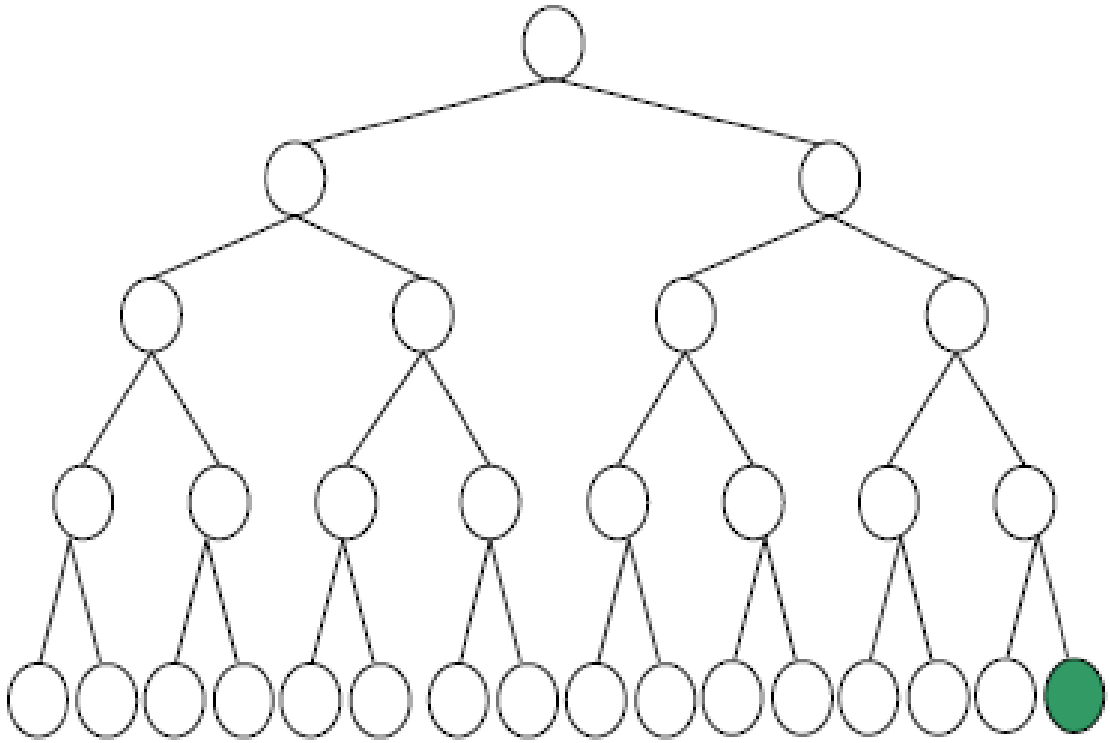




Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Greedy best-first search

- Not complete (unless m is finite; uncommon in trees).
- May get stuck in an infinite loop (e.g., reaching a dead end through a reversible action).
- Not optimal.
- Time complexity = $O(b^m)$.
- Space complexity = $O(b^m)$.



GBFS properties:

Upper-bound case: goal is last node of the tree:

Number of nodes generated: b nodes for each node of m levels (entire tree).

Time and space complexity: all generated nodes $O(b^m)$.



A* search

- Uniform cost search orders the queue according to the path cost g(n):
  - ✓ Optimal, complete, but inefficient in time and space.
- Greedy best first search orders the queue using the heuristic cost h(n):
  - ✓ Not optimal, not complete but efficient and directed (with good heuristic).

A* search

- Idea behind A* is to combine the two strategies:
  - Use an evaluation function f (n) = g(n) + h(n) to order the nodes to be explored.
  - f (n) measures the cheapest total estimated cost from the initial state to the goal state passing through the current state n.
- The resulting search is both optimal and complete assuming certain conditions on the heuristic cost h(n).
- A* algorithm is identical to UNIFORM-COST-SEARCH except that g + h is used instead of g.

A* search algorithm (tree version)

```
function Astrat-SEARCH(problem)
node ← STATE=problem:INITIAL-STATE, PATH-COST=0
frontier ← priority queue ordered by evaluation function f
loop do
if frontier.EMPTY?() then return failure
node ← frontier.POP() /* choose lowest-cost node in frontier */
if problem.GOAL-TEST(node.STATE) then return node.SOLUTION()

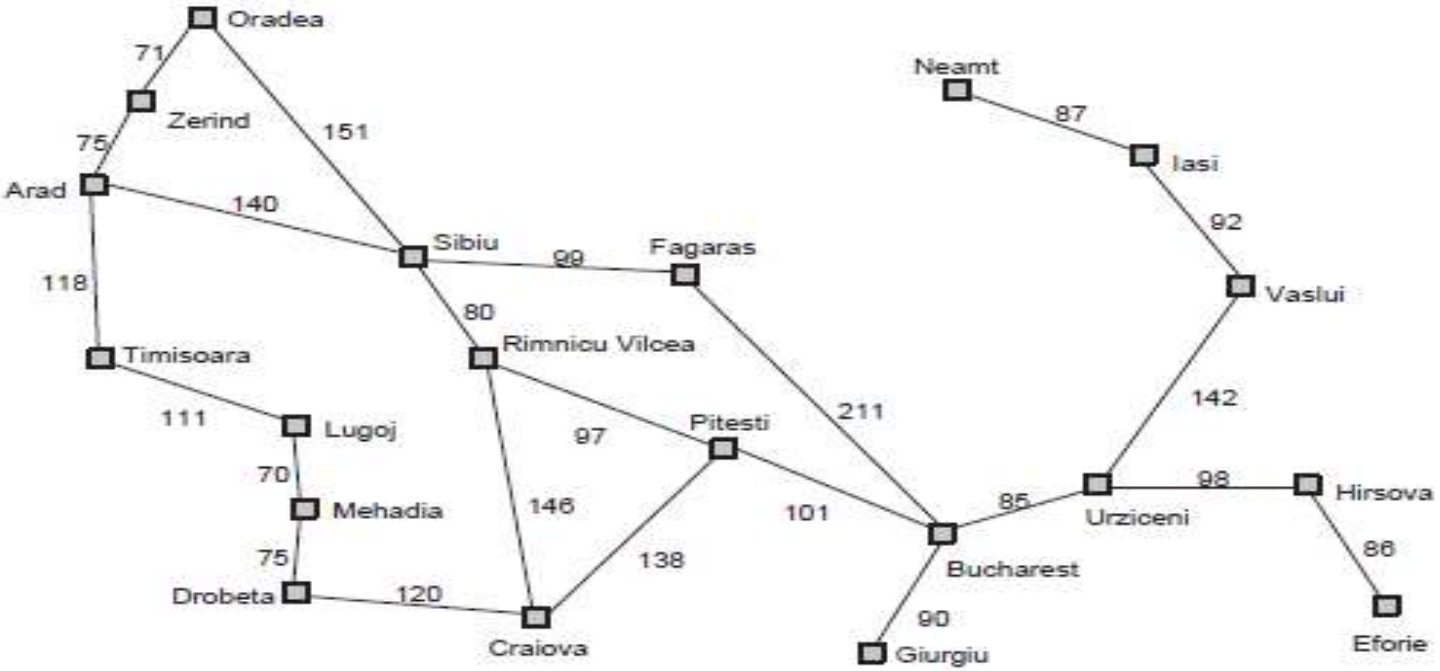
for each action in problem.ACTIONS(node.STATE) do
child ← node.CHILD-NODE(problem, action)
Frontier.INSERT(child, f(child) )
```

A* search algorithm (Graph version)

```
function GREEDY-BEST-FIRST-SEARCH(problem)
node ← STATE=problem:INITIAL-STATE, PATH-COST=0
frontier ← priority queue ordered by evolution function f
explored ← an empty set
loop do
if frontier.EMPTY?() then return failure
node ← frontier.POP() /* choose lowest-cost node in frontier */
add node.STATE to explored
if problem.GOAL-TEST(node.STATE) then return node.SOLUTION()
for each action in problem.ACTIONS(node.STATE) do
child ← node.CHILD-NODE(problem, action)
if child.STATE is not in explored and not in frontier then
        Frontier.INSERT(child, f(child) )
else if child:STATE is in frontier with higher h-value then
 replace that frontier node with child
```

A* example - Romania map

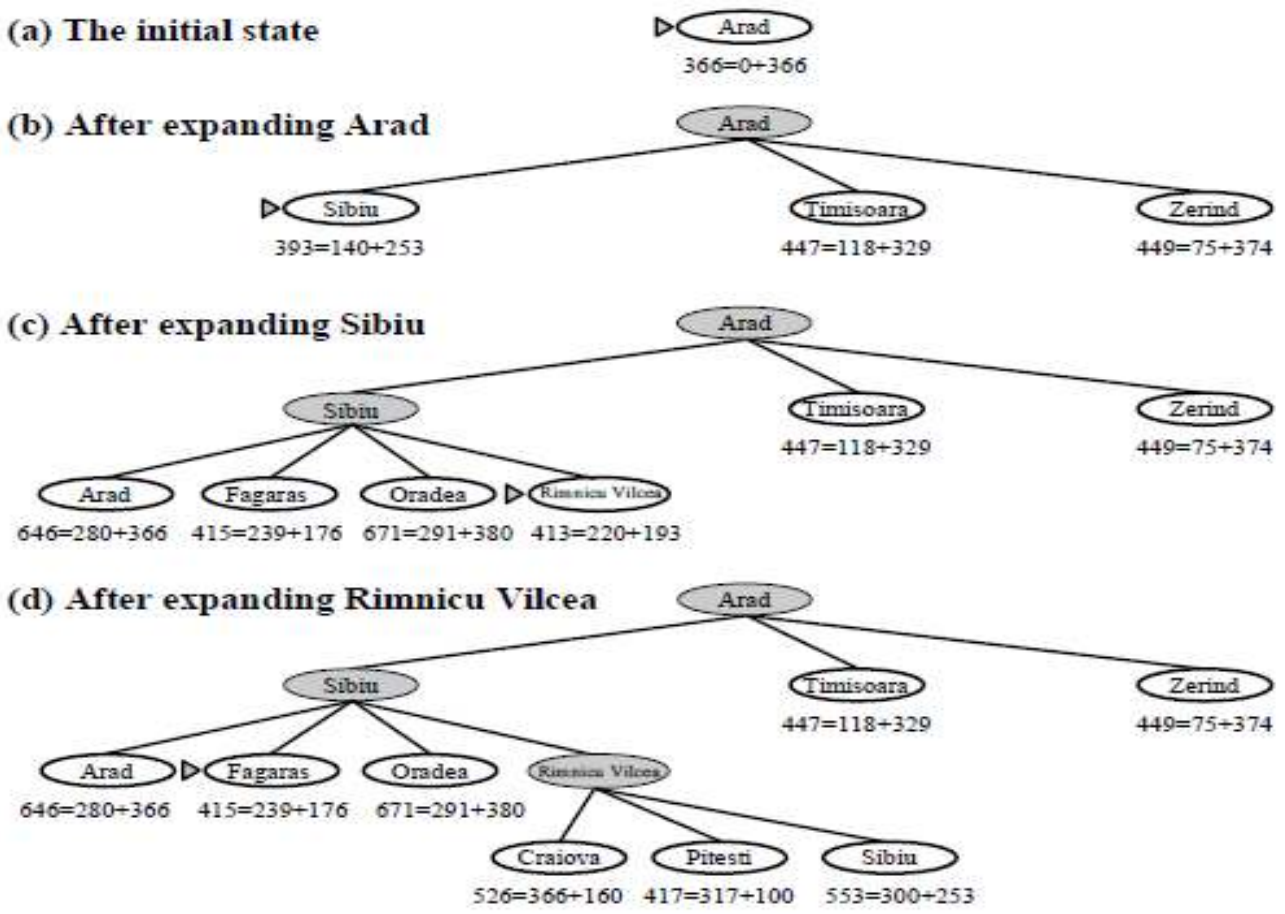
Find a route from Arad to Bucharest.



straight-line distances to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

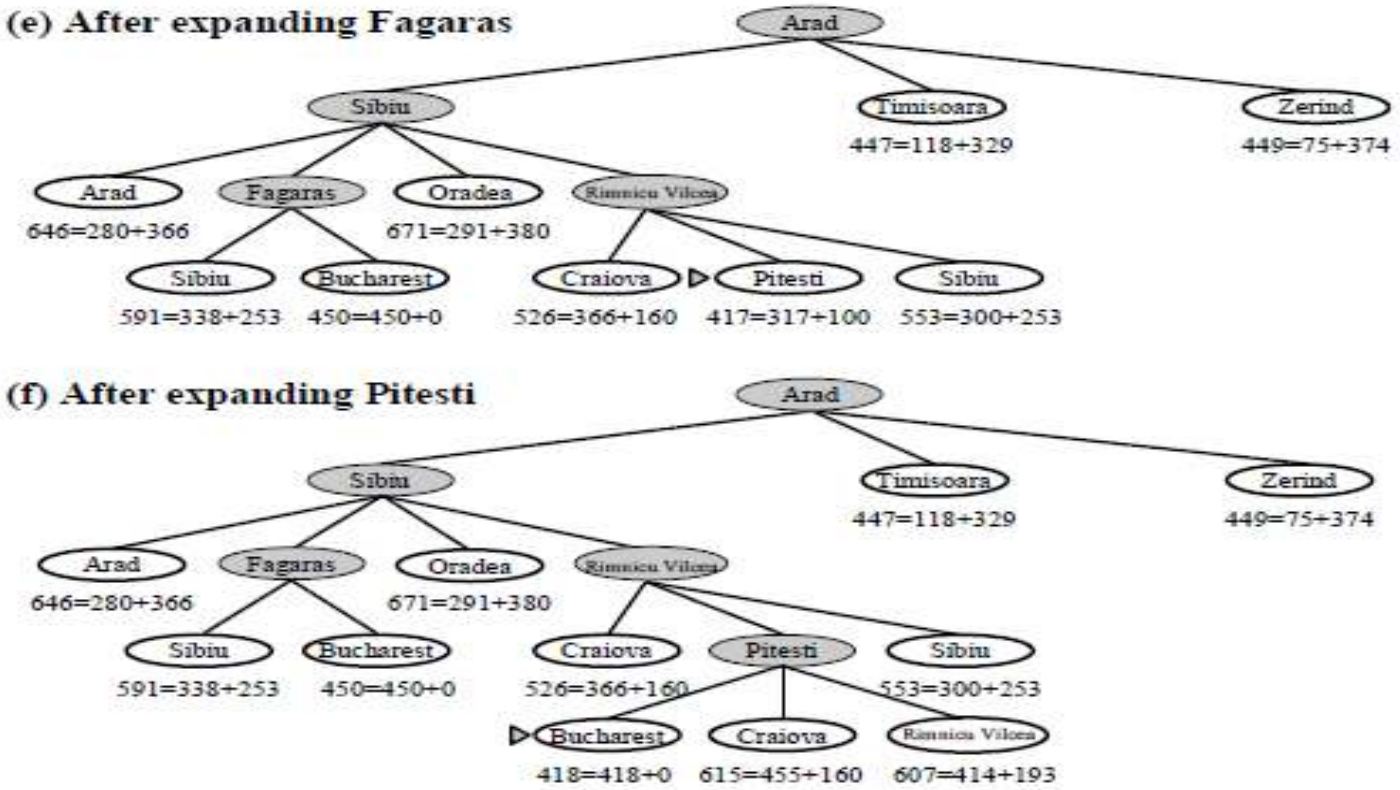
A* example - Romania map

Find a route from Arad to Bucharest.



A* example - Romania map

Find a route from Arad to Bucharest.



- Complete (if b and " are finite).
  - Reason: number of nodes with cost C is finite.
- Optimal, given an admissible heuristic.
- Time and space complexity: not straightforward!
  - Number of nodes explored depends on the difference between h and h* (true cost).
  - If h = h*, A* expands only the nodes on the optimal solution path(s).
  - If h = 0, A* consumes as much (time/space) resources as UCS.

- A heuristic h(n) is admissible if for every node n:
  - $h(n) \leq h^*(n)$,
- where $h^*(n)$ is the true cost to reach goal state from n.

- An admissible heuristic function h(n) **never overestimates the true cost from n to goal.**

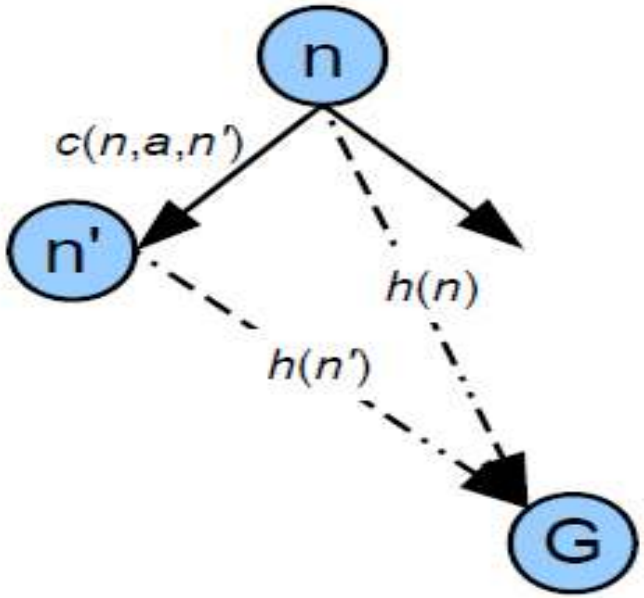
- If h(n) never overestimates, then f (n) never overestimates the true cost to the goal through node n.

- In graph-search algorithms, a new node is discarded if it's already in the explored set.
- If the new node has a better path cost (g) than the old node, a shorter path has been ignored.
- This means that the graph version of A* search is not optimal any more!
- To ensure optimality, A* must use a **consistent** heuristic function h.
  - Consistency is a stronger condition than admissibility.

A* search

- **Consistency** (monotonicity) means that for every node n and child node n' reachable from n by action a, the estimated cost $h(n)$ is never greater than the estimated cost $h(n')$ plus the step cost of getting to n':

$$h(n) \leq h(n') + c(n, a, n')$$



A* search

- A consistent heuristic function $h(n)$ **never overestimates the true cost of action a taken from n to n'**:

$$h(n) - h(n') \leq c(n, a, n')$$

- With some mathematical arrangements:

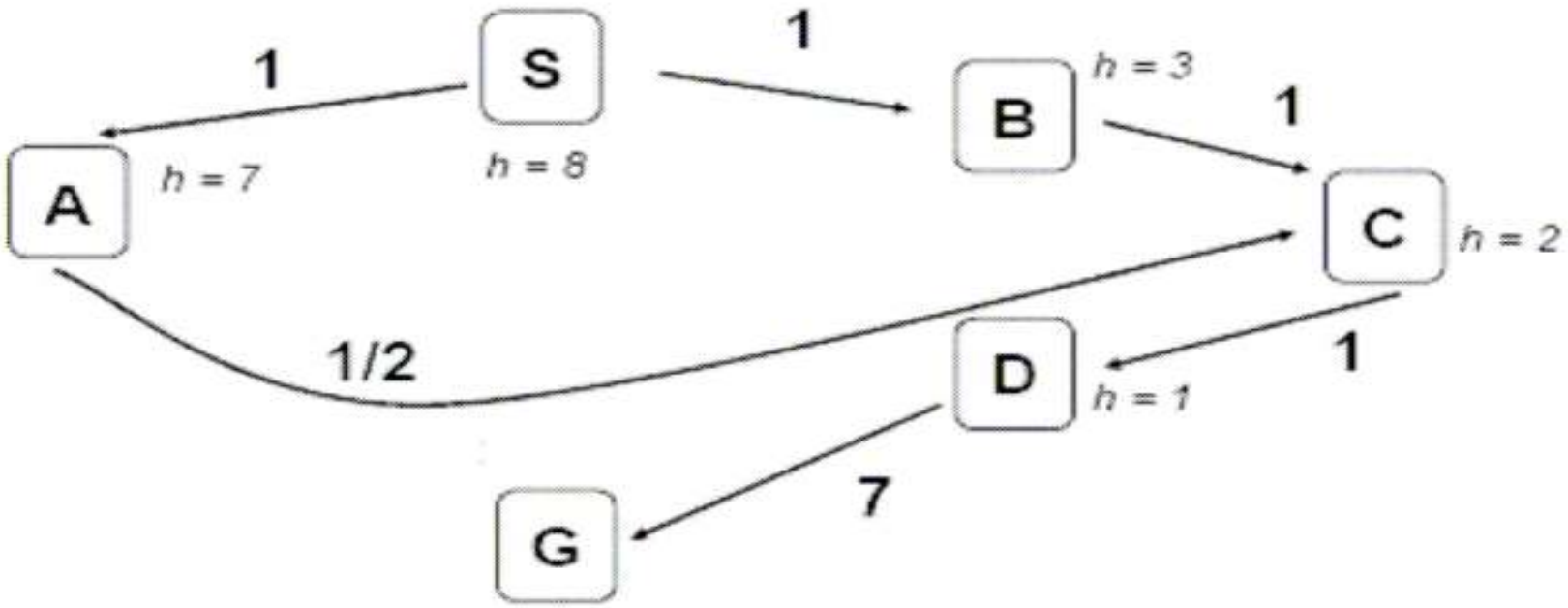
$$\begin{aligned} f(n) &= h(n) + g(n) \\ &\leq h(n') + c(n, a, n') + g(n) \\ &\leq h(n') + g(n') \\ &\leq f(n') \end{aligned}$$

So, $f(n)$ never decreases as we approach the goal.

- Consistency guarantees that states are always visited by the cheapest path first; no need to check if subsequent paths are better than the first.

A* example - consistency

Check whether h is consistent, and perform A* search.



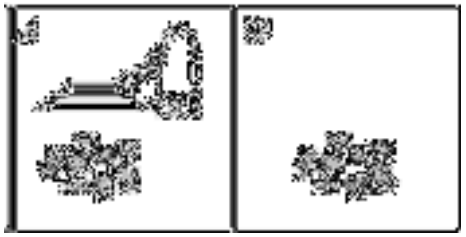
Admissible guarantee optimal solution in A* tree version ?

Consistence guarantee optimal solution in A* Graph version ?

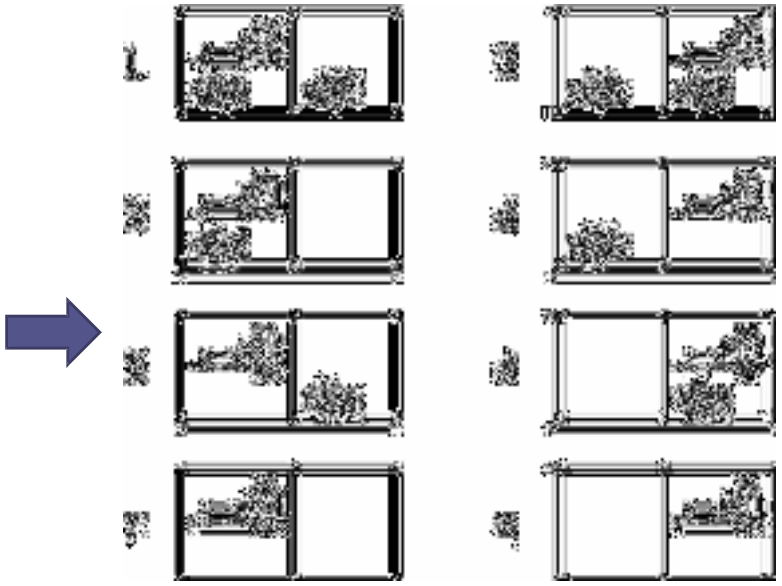
Search for the answer and prepare your self next section for this Question ?

State Space

Vacuum cleaner world

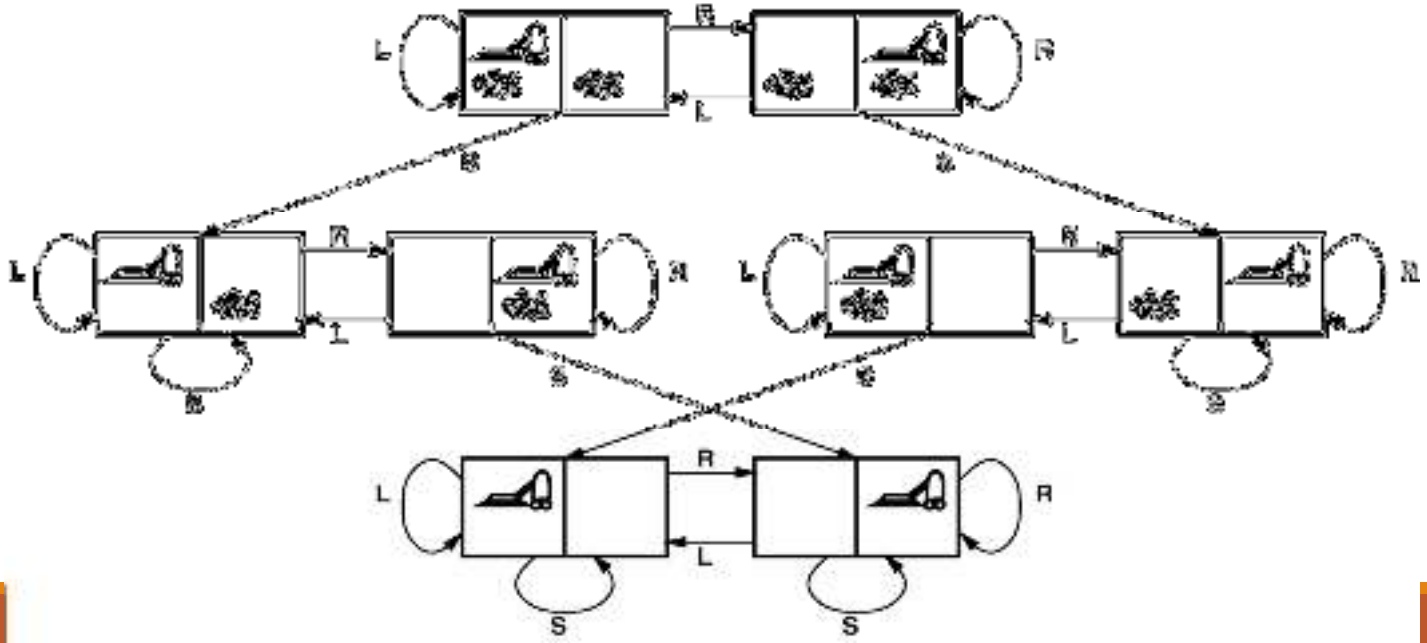


$2 * 2^8$

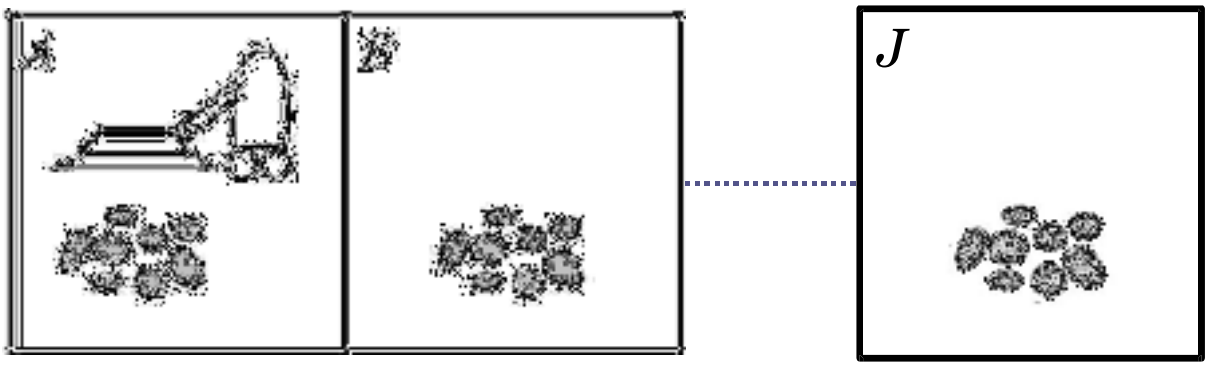


Vacuum cleaner state space

- Goal formulation: intuitively, we want all the dirt cleaned up. Formally, the goal is { state 7, state 8 }.
- Problem formulation: we already know what the set of all possible states is. The actions/operators are "move left", "move right", and "vacuum".



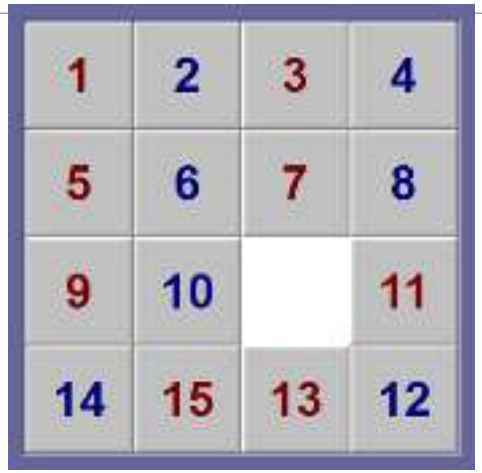
Why we need search techniques?



- Power (On / Off / Sleep)
- Camera (On /Of)
- Brush Height (1, 2, 3, 4, 5)

- States=$10 * 2^{10} * 3 * 2 * 5$ (Large state space)

Sliding blocks puzzle



- h_1= no. of misplaced blocks
- $h_2=\sum_{i=1}^{15}(no.\ distances\ for\ i\ to\ the\ correct\ place)$

Sliding blocks puzzle



Which one is admissible ?

- ☑ h_1= no. of misplaced blocks
- ☑ $h_2=\sum_{i=0}^{15}(no.\ distances\ for\ i\ to\ the\ correct\ place)$

$h_1 < h_2$, thus h_2 expand fewer pathes

Where is the Intelligence ?

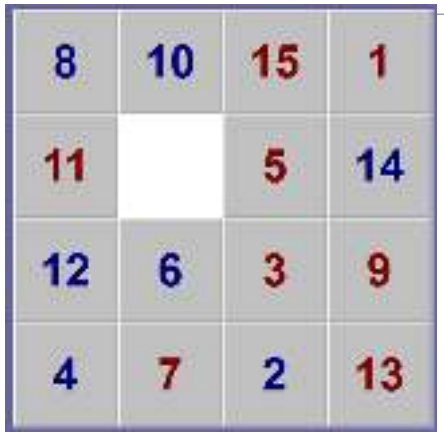


- h_1= no. of misplaced blocks
- $h_2=\sum_{i=0}^{15}$ (no. distances for i to the correct place)

- The intelligence if the agent was able to automatically find a good heuristic function.
- e.g. the agent know that: a block can move from A to B

- ~~If (A adjacent to B)~~ h_1
- ~~If (B is blank)~~ h_2

Where is the Intelligence ? cont.



- h_1= no. of misplaced blocks
- $h_2=\sum_{i=0}^{15}$ (no. distances for i to the correct place)

- So heuristic rules can be derived automatically from the formal description of the problem.
- A good heuristic would be
  - $h = \max(h_1, h_2)$

What do I need from you

When given a certain problem you should be able to:

- Build the search tree up to a given depth.
- Traverse the search tree according to a given strategy.
- Indicate whether a given heuristic is
- admissible/consistent or not.

Answer descriptive questions.